Python Execution Model and Bytecode Compilation

Source URL: https://docs.python.org/3/c-api/
License: Python Software Foundation License (PSFL)

1. Bytecode Compilation
When a Python script is executed, the CPython reference interpreter does not execute raw source code directly. Instead, the parser first translates the Python source (.py) into an intermediate, lower-level representation called Python bytecode.

Bytecode instructions are compact sequences of operation codes (opcodes) designed to be executed by the Python Virtual Machine (PVM). Compiled bytecode files are cached on disk within the __pycache__ directory using the .pyc extension to accelerate subsequent startup times. If the source file has not been modified, Python loads the .pyc file directly, bypassing tokenization, parsing, and compilation.

2. The Python Virtual Machine (PVM)
The Python Virtual Machine is the runtime execution engine loop that evaluates bytecode instructions sequentially. The PVM is a stack-based abstract machine that pushes operands onto a value stack, performs operations, and pops results. The standard dis module can be used to inspect the disassembled bytecode instructions for any function or code object.

Python Memory Management and Garbage Collection

Source URL: https://docs.python.org/3/c-api/memory.html
License: Python Software Foundation License (PSFL)

3. Reference Counting
In CPython, the primary memory management mechanism is reference counting. Every Python object contains a header field named ob_refcnt that records the number of active references pointing to that object.
When an object reference is assigned or passed to a function, its reference count increments. When a reference goes out of scope or is explicitly deleted using the del statement, its reference count decrements. When ob_refcnt reaches zero, the memory occupied by the object is deallocated immediately.

4. Cyclic Garbage Collection
Reference counting alone cannot reclaim cyclic references where two or more objects refer to each other (for instance, a parent node pointing to a child node and the child pointing back to the parent) while becoming unreachable from the root scope.
To handle reference cycles, CPython incorporates a generational cyclic garbage collector (gc module). The garbage collector classifies objects into three generations (Generation 0, 1, and 2) based on how many collection cycles they survive. Generation 0 contains newly allocated objects and is inspected most frequently.

5. The Global Interpreter Lock (GIL)
CPython utilizes a mutex known as the Global Interpreter Lock (GIL) to synchronize thread execution and protect internal interpreter state, particularly reference counts, from race conditions. Because of the GIL, only one native thread can execute Python bytecode at a time within a single OS process. For CPU-bound parallel workloads, Python developers typically employ the multiprocessing module or subinterpreters rather than multithreading.